

Skeleton Proposal for Thread-Local Storage (TLS)

ISO/IEC JTC1 SC22 WG21 P0108R1 - 2016-04-14

Paul E. McKenney, paulmck@linux.vnet.ibm.com

JF Bastien, jfb@google.com

Audience: SG1, LEWG

Introduction

This document is a revision of [P0108R0](#) based on discussions in the SG1 study group in October 2015 in Kona. P0108R0 was in turn a follow-on to [N4376](#), and provides an initial description of a potential solution to the TLS problem statement implied by that document.

Summary of Problem Statement

We expect that lightweight executors will have problems with TLS as currently envisioned and implemented. For example, some types of executors nest hierarchically, so that a number of light-weight executors might run in the context of a single heavy-weight `std::thread`. If a given function accesses TLS, and is called both from the context of a `std::thread` and from the context of a task executing within an `std::thread`, what should its TLS accesses do? If the instances invoked from a task access task-level TLS data, the function must do different things when invoked in different contexts. If the `std::thread`-level TLS data is accessed, then the task-level accesses might introduce data races and thus undefined behavior.

This also can interact with signal handling. To see this, suppose that a signal arrives at a `std::thread` while that `std::thread` is running a light-weight executor, for example, a task. The signal handler will likely conceptually be part of the `std::thread` rather than the task. This would imply some additional context switching at signal-handler start and end.

TLS is most especially a problem for light-weight executors implementing same-instruction-multiple-data (SIMD) units and general-purpose graphical processing units (GPGPUs) because large programs can have very large amounts of TLS data, each item of which might have C++ constructors and destructors. Spending many milliseconds to run constructors and destructors for a SIMD computation that only takes a few microseconds to run is clearly not a reasonable way to achieve high performance. The use of lazy construction reduces this overhead, but the occasional “vacation” taken from processing to run some constructor might be quite unwelcome.

GPGPU code often has longer runtimes, but they also tend to run extremely large numbers of threads, adding a memory-footprint problem to the constructor-destructor overhead problem. To make matters worse, in some environments, the constructors and destructors must be run on heavyweight CPUs rather than on the lightweight GPGPU hardware threads, which severely restricts the computational resources that can be applied to run constructors and destructors for GPGPU TLS data.

At the source-code level, it isn't generally knowable which executor a function is called from, or even if a function is called from multiple executors. It is left up to the programmer to write code which correctly accesses state for the executor(s) that the code will execute in. (In theory, we could of course use a TLS variable to record what type of executor was currently executing, but in practice that of course requires a TLS implementation that is efficient enough to be used by light-weight executors, and if we had that, we wouldn't be writing this paper.)

Tentative Goals

There are a number of possible ways of resolving this issue, as discussed in [N4376](#), however, this paper focuses on the possibility that TLS is an optional component of an executor. With this approach, `std::thread` implements TLS, but lighter-weight executors might choose not to. At a minimum developers intending to target lighter-weight executors may choose to author code which doesn't use TLS, thereby avoiding performance pitfalls or lack of support on those executors. The current Standard Library unfortunately implicitly uses TLS in a variety of places, making TLS avoidance difficult.

For this approach, we put forward the following tentative goals:

1. Make TLS availability optional for light-weight executors, as noted above.
 1. Provide new Standard Library functionality which avoids using TLS.
 2. Offer a clear migration path from older versions of these library APIs.
 3. Maintain the performance and scalability of high-quality standard-library implementations.
2. Avoid source-code changes for existing code running in existing executors (such as `std::thread`) that provide TLS.

3. Avoid the need to recompile existing code running in existing executors (such as `std::thread`) that provide TLS.
4. Recruit sanitizer developers to help identify issues in new code and in standard-library code related to this change.

The next section exercises these goals by attempting to apply them to the TLS `errno` facility as used by the standard math library, in the hope of sparking productive discussion. Note that when multiple lightweight executors run concurrently in the context of a single `std::thread`, setting `errno` implicitly (and for some, surprisingly) invokes undefined behavior, so a fix is a matter of some importance. At a minimum, lightweight executors that do not support TLS need to state that attempts to access TLS results in undefined behavior.

The Curious Case of `errno` and the Standard Math Library

C++ provides a per-`std::thread` facility named `errno` (19.4) in order to provide POSIX compatibility. This is also required to allow C++'s standard math library (26) maintain compatibility with that of C. Section 7.12 of the C standard specifies that `math_errhandling & MATH_ERRNO` being non-zero indicates that certain errors are available via `errno`. Furthermore, Section 19.4 of the C++ standard specifies that `errno` is provided on a per-thread basis. Therefore, `errno` is frequently implemented using TLS, which in turn means that the math library's use of `errno` forms an excellent initial test case for changes to TLS.

Preferred Approach: `status_value`

The preferred approach is to provide alternative wrappers for the functions in a new namespace, so that `errno`-oblivious code could simply assign the return value to a variable, relying on user-defined conversions to take up the slack. The hope is that LEWG's proposed [status_value](#) can serve this purpose, since it would make math functions look the same as other APIs being considered for inclusion in the Standard.

To truly "feel" the same as current `errno`-using math functions, the `status_value` class would need to be extended with an implicit conversion to return its `Value`, which we'll suggest to LEWG. It was also suggested that floating-point math functions could store their error status using NaN bits, which the current `status_value` design forbids (it must be able to hold both a status *and* a value).

Code sensitive to `errno` could assign the return value to a `status_value` and then extract both the `errno` and the return value.

This approach allows `errno`-ignoring code to run safely in light-weight executors, with modest changes for code that pays attention to `errno`. One way of preventing silent miscomputation by `errno`-ignoring code is to use exceptions, which `status_value` also supports.

Alternative Approach: `math_result`

If adding an implicit conversion to `status_value` to return its `Value` proves infeasible, it is of course easy to create a return type solely for the use of the math library. We expect LEWG to provide guidance in this area. The following fanciful code defines a new class `math_result` for this purpose:

```
1 namespace std {
2 namespace experimental {
3
4 enum class math_error {
5     divide_by_zero,
6     inexact,
7     overflow,
8     underflow,
9     invalid,
10    // ...
11 };
12
13 template <typename T>
14 class math_result {
15     math_error e;
16     T val;
17
18 public:
19     explicit operator math_error() const { return e; }
20     operator T() const { return val; }
21 };
22
23 math_result<float> tgamma(float x) {
24     math_result<float> ret;
25
26     ret.val = tgammaf(x, &ret.e);
27     return ret;
28 }
29
30 math_result<double> tgamma(double x) {
```

```

31  math_result<double> ret;
32
33  ret.val = tgamma(x, &ret.e);
34  return ret;
35 }
36
37 } // namespace experimental
38
39 template <>
40 struct is_error_code_enum<experimental::math_error> : std::true_type {};
41
42 } // namespace std

```

Paths Not Taken

The following sections serve as tombstones for approaches described in earlier versions of this paper that are now deprecated.

1. Restricting configuration.
2. Adding `errno` parameter via function overloading.
3. Limit TLS via a class-hierarchy-like approach.
4. Use IEEE NaNs or other machine state to record errors.

Restricting Configuration

One approach is to require that `math_errhandling` & `MATH_ERREXCEPT` be non-zero (as is required for IEC 60559) and that `math_errhandling` & `MATH_ERRNO` be zero in all cases where math library functions are invoked from executors that do not provide TLS. Note that `math_errhandling` is global and constant, which means that it cannot have different values in different contexts of the same execution. However, this approach cannot be used in conjunction with existing code that invokes math functions and tests `errno`. This could in turn be dealt with by forbidding use of code that checks for math errors using `errno`, but this would have the undesirable effect of acting as a barrier to the adoption of light-weight executors. It also makes it difficult to check for math errors at all.

Adding `errno` Parameter Via Function Overloading

Another approach is to use function overloading, so that an additional `double sqrt(double, int *)` declaration could be used in light-weight executors. Note that in some implementations this could require modifying the underlying C library in order to bypass `errno` setting. Code invoked both from light-weight and heavy-weight executors would need to use the new declaration, but code invoked only from heavy-weight executors could continue using the old API, consistent with the goals preserving existing source and binary code. It is tempting to instead overload on the return value, but C++ of course does not support this notion. A (probably partial) list of new APIs is as follows:

- `double acos(double x, int *errnm);`
- `float acosf(float x, int *errnm);`
- `long double acosl(long double x, int *errnm);`
- `double asin(double x, int *errnm);`
- `float asinf(float x, int *errnm);`
- `long double asinl(long double x, int *errnm);`
- `double atan2(double y, double x, int *errnm);`
- `float atan2f(float y, float x, int *errnm);`
- `long double atan2l(long double y, long double x, int *errnm);`
- `double acosh(double x, int *errnm);`
- `float acoshf(float x, int *errnm);`
- `long double acoshl(long double x, int *errnm);`
- `double atanh(double x, int *errnm);`
- `float atanhf(float x, int *errnm);`
- `long double atanh1(long double x, int *errnm);`
- `double cosh(double x, int *errnm);`
- `float coshf(float x, int *errnm);`
- `long double coshl(long double x, int *errnm);`
- `double sinh(double x, int *errnm);`
- `float sinh1(long double x, int *errnm);`
- `long double sinhl(long double x, int *errnm);`
- `double exp(double x, int *errnm);`
- `float expf(float x, int *errnm);`
- `long double expl(long double x, int *errnm);`

- double exp2(double x, int *errnm);
- float exp2f(float x, int *errnm);
- long double exp2l(long double x, int *errnm);
- double expm1(double x, int *errnm);
- float expm1f(float x, int *errnm);
- long double expm1l(long double x, int *errnm);
- int ilogb(double x, int *errnm);
- int ilogbf(float x, int *errnm);
- int ilogbl(long double x, int *errnm);
- double log(double x, int *errnm);
- float logf(float x, int *errnm);
- long double logl(long double x, int *errnm);
- double log10(double x, int *errnm);
- float log10f(float x, int *errnm);
- long double log10l(long double x, int *errnm);
- double log1p(double x, int *errnm);
- float log1pf(float x, int *errnm);
- long double log1pl(long double x, int *errnm);
- double log2(double x, int *errnm);
- float log2f(float x, int *errnm);
- long double log2l(long double x, int *errnm);
- double logb(double x, int *errnm);
- float logbf(float x, int *errnm);
- long double logbl(long double x, int *errnm);
- double scalbn(double x, int n, int *errnm);
- float scalbnf(float x, int n, int *errnm);
- long double scalbnl(long double x, int n, int *errnm);
- double scalbln(double x, long int n, int *errnm);
- float scalblnf(float x, long int n, int *errnm);
- long double scalblnl(long double x, long int n, int *errnm);
- double hypot(double x, double y, int *errnm);
- float hypotf(float x, float y, int *errnm);
- long double hypotl(long double x, long double y, int *errnm);
- double pow(double x, double y, int *errnm);
- float powf(float x, float y, int *errnm);
- long double powl(long double x, long double y, int *errnm);
- double sqrt(double x, int *errnm);
- float sqrtf(float x, int *errnm);
- long double sqrtl(long double x, int *errnm);
- double erfc(double x, int *errnm);
- float erfcf(float x, int *errnm);
- long double erfcl(long double x, int *errnm);
- double lgamma(double x, int *errnm);
- float lgammaf(float x, int *errnm);
- long double lgammal(long double x, int *errnm);
- double tgamma(double x, int *errnm);
- float tgammaf(float x, int *errnm);
- long double tgammaal(long double x, int *errnm);
- long int lrint(double x, int *errnm);
- long int lrintf(float x, int *errnm);
- long int lrintl(long double x, int *errnm);
- long long int llrint(double x, int *errnm);
- long long int llrintf(float x, int *errnm);
- long long int llrintl(long double x, int *errnm);
- long int lround(double x, int *errnm);
- long int lroundf(float x, int *errnm);
- long int lroundl(long double x, int *errnm);
- long long int llround(double x, int *errnm);
- long long int llroundf(float x, int *errnm);
- long long int llroundl(long double x, int *errnm);
- double fmod(double x, double y, int *errnm);
- float fmodf(float x, float y, int *errnm);

- `long double fmodl(long double x, long double y, int *errnm);`
- `double remainder(double x, double y, int *errnm);`
- `float remainderf(float x, float y, int *errnm);`
- `long double remainderl(long double x, long double y, int *errnm);`
- `double remquo(double x, double y, int *quo, int *errnm);`
- `float remquof(float x, float y, int *quo, int *errnm);`
- `long double remquol(long double x, long double y, int *quo, int *errnm);`
- `double nextafter(double x, double y, int *errnm);`
- `float nextafterf(float x, float y, int *errnm);`
- `long double nextafterl(long double x, long double y, int *errnm);`
- `double fdim(double x, double y, int *errnm);`
- `float fdimf(float x, float y, int *errnm);`
- `long double fdiml(long double x, long double y, int *errnm);`
- `double fma(double x, double y, double z, int *errnm);`
- `float fmaf(float x, float y, float z, int *errnm);`
- `long double fmal(long double x, long double y, long double z, int *errnm);`

Note that new APIs need be provided only for those math functions that set `errno`. Note also that because C does not provide function overloading, different names will need to be used should C adopt similar functionality.

One might expect some dissatisfaction with the invention of more than 100 new functions, especially given that a great many uses of these functions ignore `errno`. Although one can argue that ignoring `errno` is a bad idea, one might also expect strenuous objections to pointless modifications of existing `errno`-ignoring code.

Machine Registers: The Ultimate TLS Implementation

The logically extreme TLS implementation is a reserved machine register, so extreme `errno` would simply reserve a register for `errno`. If common code was to be invoked from both light-weight and heavy-weight executors, a simple solution is to always reserve a register for `errno`.

Of course, this approach simply does not scale with increasing numbers of TLS objects, as even modern machines have a rather limited number of registers. In addition, some TLS constructors might not react well to finding that all of their data was in machine registers, especially those constructors expecting to create linked structures. Nevertheless, this approach might work well for restricted quantities of TLS data, such as that which might be needed for a non-hosted small-library implementation.

Applying Lessons from Class Hierarchies

Suppose that we (very loosely) modeled TLS data with a class-like hierarchy. The “base class” would contain only that TLS data required by the core language, and “subclasses” would add TLS data required by libraries and by the user application. Ignoring the analogies with abstract classes for the moment, light-weight executors might confine themselves to TLS data relatively high up in this TLS hierarchy, while heavy-weight executors might take the entire class hierarchy, lock, stock, and barrel.

Any executor expecting to use the math library would need to maintain that part of the TLS hierarchy containing `errno`. Future work might identify a minimum subset for various types of executors.

Use Hardware State (NaNs) to Record Errors

IEEE floating-point not-a-number (NaN) values were designed to record error conditions and to flow them through the remainder of the computation. Non-IEEE hardware often has other facilities for this purpose, and some IEEE hardware has cheaper ways to maintain this information.

The key point here is to provide an API to read this information out. The API must take a floating-point number as input for the NaN case, however, implementations that do not use NaNs are free to ignore this number and instead read out hardware state. The return value is an integer `errno`.

```

1 int recent_errno(float x);
2 int recent_errno(double x);
3 int recent_errno(long double x);
4
5 extern "C" {
6     int recent_fp_errno(float x);
7     int recent_dp_errno(double x);
8     int recent_ldp_errno(long double x);
9 }
```

This approach only applies to functions that return a floating-point number. Functions that return integral types (integer log

functions and round-to-integer functions) must use some other alternative to stop using `errno`.

Note that NaNs can be used, if desired, in conjunction with either the preferred `status_value` approach or the alternative `math_result` approach.

Additional Information

Floating-point state is stored on a per-thread basis, which means that if a light-weight executor can be preempted or migrated among `std::thread` instance, things like rounding modes and error/exception indications can be subject to unscheduled revision.